Relatório Técnico de Laboratório

□ Relatório Técnico de Laboratório

Informações do grupo

- □ **Curso:** Engenharia de Software
- □ **Disciplina:** Laboratório de Experimentação de Software
- □ **Período:** 6° Período
- □ **Membros do Grupo:** Júlia Medeiros e Thiago Laass

---

1 Introdução

1.1 Contextualização

Este laboratório analisa características de repositórios open-source populares no GitHub, tomando como base os 1000 repositórios de código com maior número de estrelas no momento da coleta. O objetivo é caracterizar esses projetos a partir de indicadores de maturidade, atividade recente, histórico de pull requests aceitas, releases, linguagem principal e resolução de issues.

1.2 Problema foco do experimento

Repositórios populares são, de fato, mais maduros e bem mantidos? Em particular, deseja-se investigar se esses projetos tendem a ser mais antigos, se acumulam volume relevante de pull requests aceitas, se apresentam histórico expressivo de releases, se permanecem ativos ao longo do tempo e se mantêm boa taxa de fechamento de issues.

1.3 Questões de Pesquisa (RQs)

- **RQ01:** Sistemas populares são maduros/antigos?
- **RQ02:** Sistemas populares apresentam alto volume de pull requests aceitas?
- **RQ03:** Sistemas populares apresentam histórico expressivo de releases?
- **RQ04:** Sistemas populares são atualizados com frequência?
- **RQ05:** Sistemas populares são escritos nas linguagens mais populares?
- **RQ06:** Sistemas populares possuem alto percentual de issues fechadas? > (Opcional) **RQ07 (bônus):** Comparar RQ02–RQ04 por linguagem.

1.4 Hipóteses

- **H01 (RQ01):** Repositórios populares tendem a ser mais antigos.
- **H02 (RQ02):** Repositórios populares apresentam alto volume de PRs aceitas.
- **H03 (RQ03):** Repositórios populares possuem número elevado de releases.
- **H04 (RQ04):** Repositórios populares são atualizados frequentemente.
- **H05 (RQ05):** Linguagens populares (Python/JS/TS) são mais frequentes.
- **H06 (RQ06):** Repositórios populares têm alta taxa de issues fechadas.

1.5 Objetivos

Objetivo principal: caracterizar repositórios populares do GitHub por métricas de processo e produto, respondendo às RQ01–RQ06 com estatísticas descritivas.

Objetivos específicos: - Coletar dados dos 1000 repositórios de código mais estrelados por meio da GitHub GraphQL API - Derivar métricas como idade do repositório, dias desde o último push e percentual de issues fechadas - Calcular medianas para as RQs numéricas e contagens por linguagem principal - Produzir tabelas e gráficos para apoiar a interpretação e a discussão dos resultados

---

2 Metodologia

2.1 Passo a passo do experimento

O experimento foi dividido em quatro etapas principais: (1) coleta dos dados, (2) transformação e organização do dataset, (3) cálculo de métricas derivadas e (4) análise estatística com apoio de visualizações. Essa sequência foi adotada para responder às RQ01–RQ06 de forma reprodutível e alinhada às métricas disponíveis na GitHub GraphQL API.

Etapas

1. Definição das questões de pesquisa e das métricas correspondentes.
2. Preparação do ambiente de execução em Python, com dependências declaradas em `requirements.txt`.
3. Configuração do token de acesso do GitHub em arquivo `.env`, mantendo o valor sensível fora do repositório e documentando o formato em `.env.example`.
4. Coleta dos repositórios via GitHub GraphQL API, usando ordenação por estrelas e paginação por cursor.
5. Persistência dos dados em duas camadas: JSON bruto para rastreabilidade e CSV processado para análise.
6. Conversão e tratamento dos dados coletados, incluindo padronização de datas e cálculo de métricas derivadas.
7. Geração de estatísticas descritivas e visualizações para interpretação dos resultados.

2.2 Materiais utilizados

- GitHub GraphQL API com autenticação por token pessoal.
- Ambiente virtual Python (`venv`) para isolar dependências.
- Arquivos de apoio à reprodutibilidade: `.env.example` para documentar a variável `GITHUB_TOKEN` e `.gitignore` para evitar vazamento de credenciais.
- Dependências instaladas via `requirements.txt`, incluindo bibliotecas para configuração do ambiente, tratamento de dados e geração de gráficos.
- Artefatos gerados pela coleta e análise: JSON bruto, CSV processado, tabelas e gráficos.

2.3 Métodos utilizados

A coleta foi feita com um script próprio utilizando a GitHub GraphQL API. A busca inicial foi configurada para retornar repositórios ordenados por estrelas (`sort:stars`), em conformidade com o objetivo de estudar projetos populares. Como o experimento exigia repositórios de código, foi adotado como critério de aceite a presença de linguagem principal definida (`primaryLanguage != null`).

Durante a execução, observou-se uma limitação prática da API de busca do GitHub: a paginação não permite percorrer indefinidamente os resultados de uma única consulta, o que dificultava alcançar 1000 repositórios aceitos quando se filtrava apenas projetos com linguagem principal definida. Para contornar esse problema, a busca foi dividida em faixas de estrelas, como `stars:50000..99999` e `stars:20000..49999`, acumulando resultados de múltiplas consultas até completar a amostra desejada.

Em cada faixa, a coleta utilizou paginação por cursor com `pageInfo.endCursor` e `hasNextPage`. Os candidatos retornados pelo search eram então enriquecidos por meio de queries GraphQL compostas com aliases (`repo0`, `repo1`, etc.), executadas em lotes menores. A saída final da coleta foi persistida em dois formatos: um JSON bruto, voltado à

auditoria e rastreabilidade, e um CSV processado com uma linha por repositório, contendo colunas como `stars`, `created_at`, `pushed_at`, `primary_language`, `releases_total`, `prs_merged_total`, `issues_total` e `issues_closed_total`.

2.4 Decisões

Após a coleta, as datas em formato ISO 8601 foram convertidas para `datetime`, permitindo o cálculo das métricas derivadas. As principais métricas calculadas foram `repo_age_days`, `days_since_push` e `issues_closed_ratio`. No caso da razão de issues fechadas, foi necessário tratar explicitamente situações com `issues_total = 0`, evitando divisão por zero.

As decisões operacionais mais relevantes foram as seguintes: - **PRs aceitas = PRs com estado `MERGED`.** Essa definição foi usada como proxy de colaboração aceita pelo projeto. - **Última atualização = `pushedAt`.** Essa escolha foi feita porque `pushedAt` representa atividade real de código, ao contrário de `updatedAt`, que também pode refletir alterações apenas em metadados. - **Amostra = 1000 repositórios de código.** Foram aceitos apenas repositórios com `primaryLanguage` definida. - **Lotes conservadores de coleta.** O tamanho do batch foi mantido em valor reduzido para evitar falhas e timeout em queries muito grandes. - **Estatística principal = mediana.** Essa opção foi adotada por ser mais robusta diante de distribuições assimétricas e presença de outliers.

Na etapa analítica, para as variáveis numéricas foram utilizadas mediana e estatísticas descritivas complementares, como percentis 25 e 75. Para a variável categórica de linguagem, foram calculadas contagens absolutas e percentuais. As visualizações incluíram gráficos de barras, histogramas, boxplots, dispersão em escala log-log e heatmap de correlação, com o objetivo de tornar visíveis padrões de concentração, cauda longa e possíveis associações entre métricas.

Entre as principais dificuldades enfrentadas, destacam-se o limite prático de resultados da busca do GitHub, a instabilidade de queries muito grandes e a forte assimetria das distribuições observadas. Esses problemas foram tratados, respectivamente, com a divisão da coleta em faixas de estrelas, a redução do tamanho dos lotes e o uso de medidas robustas e visualizações com escala log quando apropriado.

2.5 Métricas e suas unidades

- **RQ01:** maturidade do projeto, medida pela idade do repositório a partir de `created_at` (dias e anos).
- **RQ02:** volume de pull requests aceitas, medido por `prs_merged_total` (contagem).
- **RQ03:** histórico de releases, medido por `releases_total` (contagem).
- **RQ04:** atividade recente, medida pelo tempo desde o último push, calculado a partir de `pushed_at` (dias).
- **RQ05:** linguagem principal do repositório, representada por `primary_language` (categoria).
- **RQ06:** taxa de fechamento de issues, calculada como `issues_closed_total / issues_total` (proporção).

---

3 Visualização dos Resultados

> Referência temporal: métricas de tempo (idade e dias desde o último push) foram calculadas com data de referência **2026-03-12 12:00 (UTC)**.

3.1 Visualização tabular (tabelas)

Tabela A — Medianas por RQ

RQ	Métrica	Mediana
RQ01	Idade do repositório	3004 dias (≈ 8,22 anos)
RQ02	Total de PRs aceitas (MERGED)	817
RQ03	Total de releases	50
RQ04	Dias desde o último push	1 dia
RQ06	Razão de issues fechadas	0,872 (≈ 87,2%)

> Fonte: tabela_mediana_rqs.csv (gerado pelo script de análise).

Tabela B — Linguagens primárias (Top 20)

Linguagem	Contagem	Percentual
Python	219	21,9%
TypeScript	181	18,1%
JavaScript	130	13,0%
Go	80	8,0%
Rust	59	5,9%
C++	54	5,4%
Java	52	5,2%
C	26	2,6%
Jupyter Notebook	26	2,6%
Shell	25	2,5%
HTML	19	1,9%
C#	12	1,2%
Ruby	12	1,2%
Kotlin	11	1,1%
CSS	9	0,9%
Swift	9	0,9%
PHP	8	0,8%
Vue	7	0,7%
Dart	6	0,6%

> Fonte: tabela_linguagens_top20.csv.

Tabela C (opcional) — Estatísticas descritivas (p25, mediana, p75)

Métrica	p25	Mediana	p75
Idade (dias)	1742	3004	4155
PRs aceitas (MERGED)	204	817	3279
Releases (total)	1	50	151
Dias desde último push	0	1	28
Razão issues fechadas	0,692	0,872	0,960

> Fonte: tabela_stats_descritivas.csv.

3.2 Visualização gráfica (figuras)

As figuras abaixo foram geradas automaticamente pelo script lab01/src/analise.py.

Figura 1 — Top 10 linguagens (contagem)

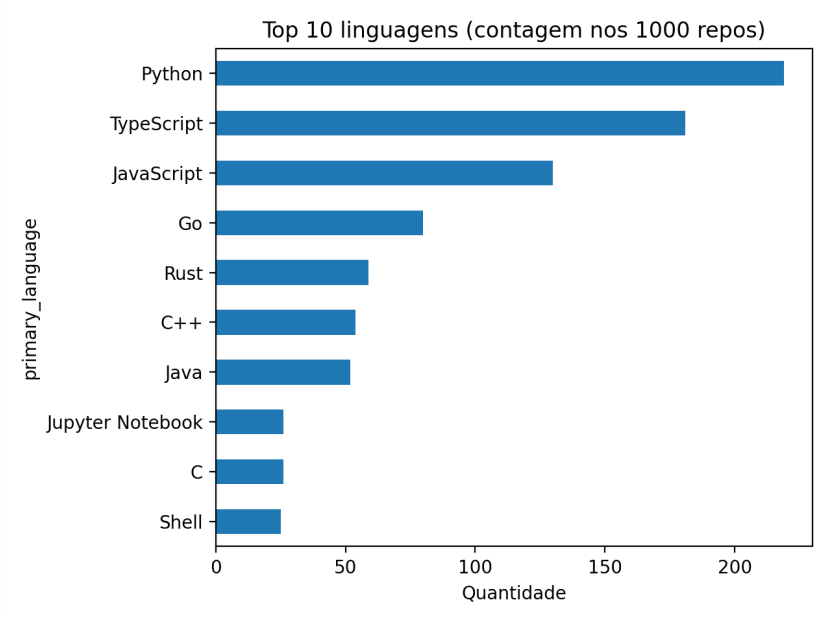


Figura 2 — Histograma da idade dos repositórios (dias)

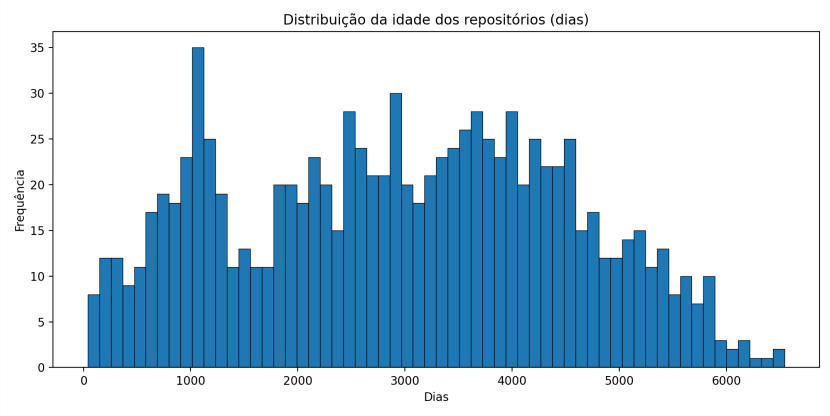


Figura 3 — Dias desde o último push (zoom até 365 dias)

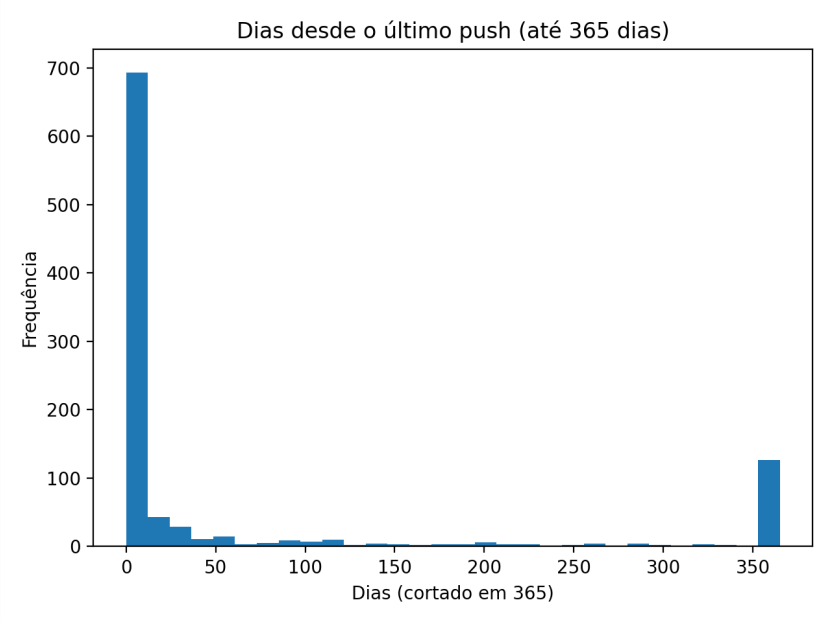


Figura 4 — Boxplot de PRs aceitas (MERGED) sem outliers

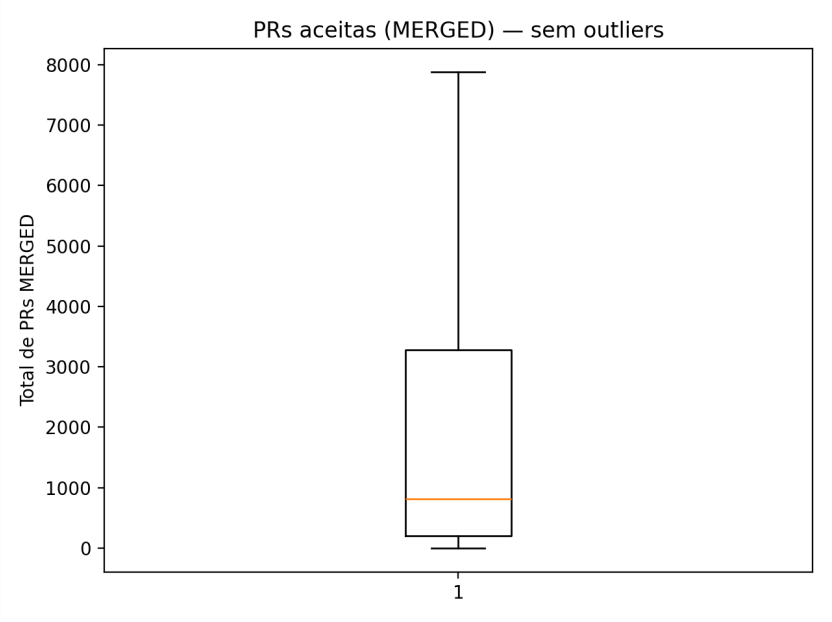


Figura 5 (opcional) — Stars vs PRs aceitas (log-log)

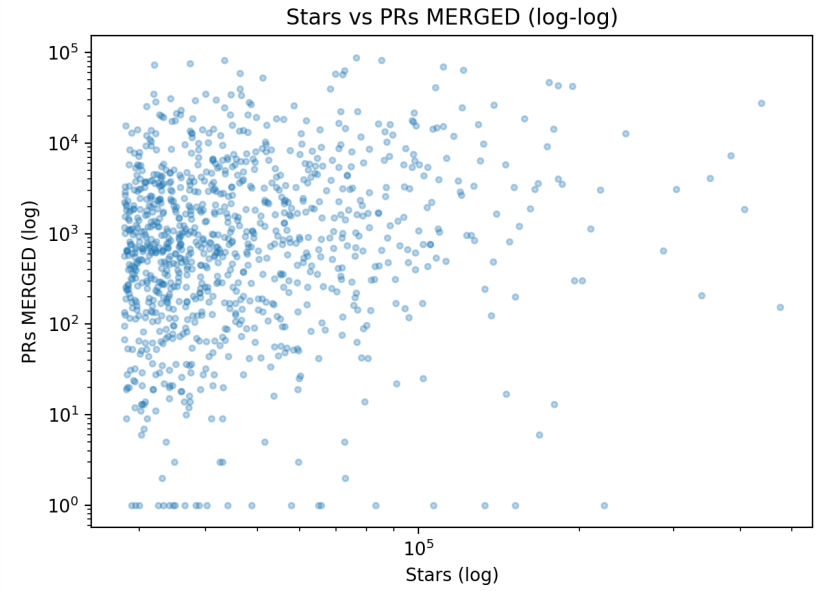
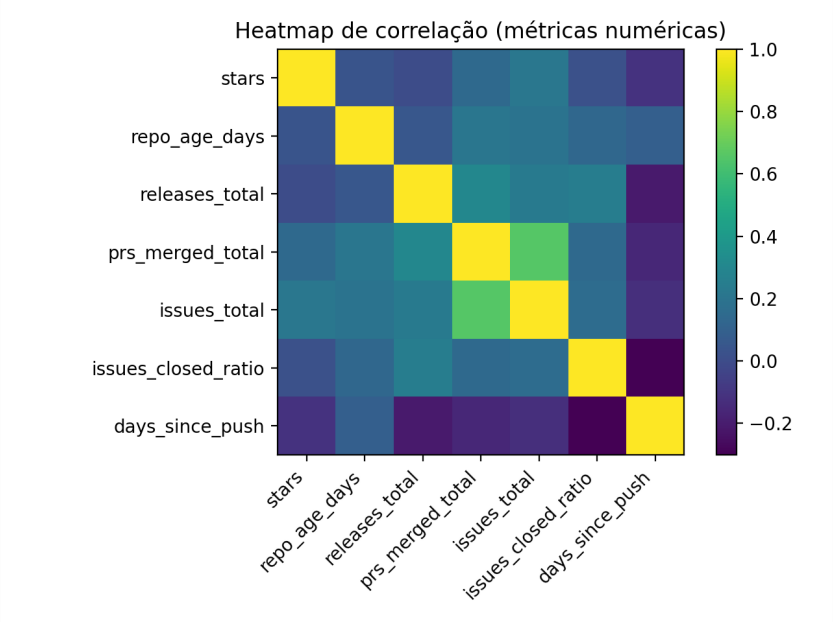


Figura 6 (opcional) — Heatmap de correlação



---

4 Discussão dos resultados

4.1 Confronto com as Questões de Pesquisa

Para cada RQ, escrever 3 coisas: - **resultado numérico (mediana / contagem) - o que isso sugere - se confirmou a hipótese**

RQ01 — Maturidade/idade - Resultado: mediana de **3004 dias** (≈ **8,22 anos**). - Interpretação: a amostra é composta majoritariamente por projetos "maduros", já estabelecidos há vários anos. Isso é coerente com o efeito de "acúmulo de reputação": projetos mais antigos tiveram mais tempo para ganhar tração e estrelas. - Hipótese (H01): **confirmada**.

RQ02 — PRs aceitas - Resultado: mediana de **817 PRs aceitas (MERGED)**. - Interpretação: mesmo usando uma métrica robusta (mediana), o volume típico de contribuição aceita é alto, sugerindo comunidades ativas e ciclos contínuos de contribuição externa. A Tabela C também indica assimetria forte (p75 bem acima da mediana). - Hipótese (H02): **confirmada**.

RQ03 — Releases - Resultado: mediana de **50 releases**. - Interpretação: projetos populares tendem a manter histórico relevante de releases, o que sugere um processo formal de empacotamento/entrega. Ao mesmo tempo, o p25=1 indica que existe uma parcela significativa que quase não faz releases (por exemplo, repositórios de conteúdo/coleções). - Hipótese (H03): **confirmada (com variação por tipo de projeto)**.

RQ04 — Frequência de atualização - Resultado: mediana de **1 dia** desde o último push. - Interpretação: a amostra está altamente concentrada em projetos com atividade muito recente (e p25=0), indicando manutenção contínua. Há outliers bem desatualizados, mas eles não dominam o "comportamento típico". - Hipótese (H04): **confirmada**.

RQ05 — Linguagens mais populares - Resultado: as linguagens mais frequentes foram **Python (21,9%)**, **TypeScript (18,1%)** e **JavaScript (13,0%)** (Tabela B). - Interpretação: isso sugere forte presença de ecossistemas web e tooling (TypeScript/JavaScript) e automação/IA/dados (Python) entre os repositórios mais estrelados. - Hipótese (H05): **confirmada**.

RQ06 — Percentual de issues fechadas - Resultado: mediana da razão de issues fechadas de **0,872** (≈ **87,2%**). - Interpretação: em geral, projetos populares mantêm uma boa taxa de resolução/fechamento, ainda que existam casos com razão próxima de 0 (possivelmente projetos com issues desabilitadas/sem uso, ou workflows que não usam issues). - Hipótese (H06): **confirmada**.

4.2 Insights (padrões observados)

- **Cauda longa**: PRs aceitas e releases apresentam forte assimetria (diferença grande entre p75 e mediana), o que é esperado em dados de popularidade/projetos "megarepos".
- **Atividade recente**: a mediana de 1 dia desde o último push sugere manutenção frequente para a maioria dos projetos.
- **Concentração por linguagem**: Python/TypeScript/JavaScript somam mais de metade da amostra (53,0%), indicando dominância desses ecossistemas no top 1000.
- **Qualidade de manutenção (proxy)**: a razão mediana de issues fechadas (~87%) sugere boa capacidade de triagem/fechamento.

4.3 Comparações e estatísticas

- O boxplot (Figura 4) evidencia que existe um conjunto pequeno de outliers com volume de PRs muito maior do que o típico.
- O scatter em log-log (Figura 5) sugere uma relação positiva entre estrelas e volume de PRs aceitas, mas com grande dispersão (projetos com muitas estrelas podem ter dinâmicas diferentes).

4.4 Bônus — RQ07 (por linguagem)

RQ07: Sistemas escritos em linguagens mais populares recebem mais contribuição externa, lançam mais releases e são atualizados com mais frequência?

Para comparação direta, foi feita uma divisão em quatro grupos: **Python**, **JavaScript**, **TypeScript** e **Outras** linguagens.

Grupo de linguagem	n	PRs MERGED (mediana)	Releases (mediana)	Dias desde push (mediana)
Python	219	620	21	2
JavaScript	130	567	34	9,5
TypeScript	181	2004	154	0
Outras	470	759	43	1

Análise (informal): - Entre as linguagens populares, **TypeScript** se destaca com medianas mais altas de PRs aceitas e releases e com atividade extremamente recente (mediana 0 dias desde push). - **Python** apresenta boa atividade recente (2 dias), mas com medianas mais baixas de releases e PRs do que TypeScript. - **JavaScript** aparece, em mediana, mais "lento" em atualização (9,5 dias) e com valores menores de PRs aceitas/releases do que TypeScript.

> Fonte: tabela_rq07_populares_vs_outras.csv.

---

5 Conclusão

5.1 Tomada de decisão (resultado conclusivo sucinto)

Com base nos resultados: - Repositórios populares tendem a ser **maduros e muito ativos**, com bons sinais de manutenção. - Evidências principais (medianas): **8,22 anos** de idade (RQ01), **1 dia** desde o último push (RQ04), **817 PRs aceitas** (RQ02), **50 releases** (RQ03) e **87,2%** de issues fechadas (RQ06). - As figuras de distribuição (idade, dias desde push e boxplot de PRs) reforçam que existe **cauda longa** (poucos projetos muito grandes), mas o comportamento típico ainda indica **alta atividade**.

5.2 Sugestões futuras

- Detalhar a RQ07 por linguagem com recortes adicionais (ex.: Top 10 linguagens, ou por "família" como sistemas vs web vs ciência de dados).
- Incluir outras métricas (ex.: forks, contributors, tamanho/LOC) se coletarem
- Analisar evolução temporal com séries mais detalhadas (por mês/ano)

5.3 Plus — relação com literatura (curto)

De forma geral, os resultados estão alinhados com observações comuns em estudos empíricos: popularidade tende a se associar a projetos mais antigos, com maior volume de contribuições e manutenção ativa, ainda que exista diversidade de tipos de repositório (software "produto", bibliotecas, coleções de links, materiais educacionais etc.).